can be interpreted a dictionary storing the associations `(0, “the”)`, `(1, “lazy”)` and `(2, “fox”)`.

Associative arrays generalize this concept, allowing keys to be drawn from virtually any possible domain.

| **Abstract Data Structure: Associative Array (aka Dictionary)** | |
|---|---|
| API | `class Dictionary {`<br>`  insert(key, value)`<br>`  remove(key) → value`<br>`  contains(key) → true/false`<br>`}` |
| Contract with client | A dictionary stores permanently all the pairs added by the client(s). If a pair (K,V) was added to the dictionary (and not removed afterwards), then `contains(K)` will return `V`. |

With this API defined, we can sketch a solution for our initial problem.

When users log in into their email, our client receives a list of contacts from the server and stores them in a *dictionary* that we can keep in memory (having so many contacts that it won’t fit the browser’s session storage would be an exceptional situation even for an Instagram rockstar!). If the user adds a new contact to our address book, we perform a call to `insert` on the dictionary; likewise, in case users remove an existing contact, we just keep the dictionary in sync by calling `remove`. Whenever a user writes an email and inserts a recipient, we first check the dictionary, and only if the contact is not in the address book, we do show a popup asking users if they would like to save the new contact.